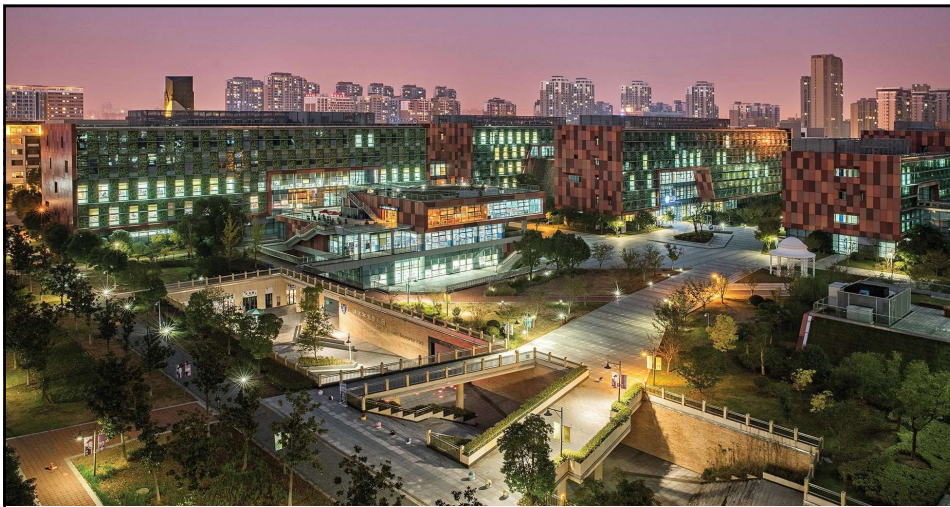




RADIAL-BASIS FUNCTION NETWORKS

INT301 Bio-computation, Week 8, 2025



1

神经网络执行的最常见任务是分类和预测。

分类 (Classification): 就是给事物“贴标签”。比如，给一张图片打上“猫”或“狗”的标签；或者判断一封邮件是“垃圾邮件”还是“非垃圾邮件”。

预测 (Prediction): 通常指“回归 (Regression)”，即预测一个连续的数值。比如，根据房屋的面积、地段来预测它的“价格”；或者预测明天的“股票价格”。

Introduction

与其像传统神经网络（如多层感知机 MLP）那样模拟“神经元激活”，RBF 网络从一开始就在数学上被设计为一个强大的“通用函数逼近器”。

神经网络的设计可以被看作是一个“曲线拟合”或“函数逼近”的问题。训练一个神经网络，本质上就是在寻找一个非常复杂的函数 f ，让这个函数 f 能够尽可能地“拟合”我们所有的训练数据 $f(\text{输入}) \approx \text{输出}$ 。

- The most common tasks performed by neural networks are **classification** and **prediction**.
- These tasks involve input-output **mappings**, and the network is designed **to learn the mapping** from knowledge of the problem environment.
- Thus, the design of a neural network can be viewed as a **curve-fitting** or **function approximation** problem. RBF 网络就是专门以“函数逼近”为首要目标的神经网络
- This viewpoint is the motivation for radial-basis function networks

2

“学习”等同于在空间中找到一个“插值表面”这个表面能“最佳拟合”训练数据

1. 插值 (Interpolating) :

在严格的数学定义里, “插值”意味着这个表面必须精确地穿过每一个训练数据点。

2. 最佳拟合 (Best Fit) :

这通常是一个更宽松、更实用的目标。它不要求表面穿过每一个点 (因为数据点可能有噪声), 而是要求这个表面尽可能地靠近所有点, 以捕捉到数据的总体趋势。

The Interpolation Problem

虽然 RBF 也能做到严格插值, 但是为了避免过拟合, 往往还是去做“最佳拟合”

“最佳拟合”的曲面, 就是那个能让这个“平均误差” (即损失函数) 最小的曲面。

- For the problem of **curve-fitting (approximation in high-dimensional spaces)**, learning is equivalent to finding an **interpolating surface in the space** that provides a best fit to the training data, measured by preselected statistical criteria.
- **Curve-fitting or interpolation problem**

3

Radial-Basis Functions

- RBF网络的核心公式。RBF旨在构建一个“插值函数” $F(x)$ 去实现一个拟合曲面
- The **radial-basis functions** technique suggests constructing of interpolation functions F of the following form:

$$F(x) = \sum_{i=1}^N w_i \phi(\|x - x_i\|)$$

where: $\phi(\|x - x_i\|)$ is a set of nonlinear radial-basis functions, x_i are the centers of these functions, and $\| \cdot \|$ is the Euclidean norm. 一个 RBF 是一个实值函数, 它的值仅仅取决于 (输入) 到原点的距离。

- A **radial basis function (RBF)** is a real-valued function whose value depends only on the distance from the origin
- $\phi(x) = \phi(\|x\|)$
- Alternative form: depends on the distance from a **center c**
取决于到某个中心点 c 的距离。 $\phi(x, c) = \phi(\|x - c\|)$
- Any function that satisfies the property is a radial basis function 只要函数的值只跟“到中心的距离”有关, 它就是 RBF。高斯函数只是其中最常用的一种。

4

$F(x)$ 网络的最终预测值

$\sum_{i=1}^N$: 一个求和符号, 表示把 N 个隐藏层神经元的个数

w_i : 第 i 个权重 weight, 代表第 i 个 RBF 隐藏神经元对最终结果的重要性或贡献度

x : 网络的输入

x_i : 第 i 个 RBF 神经元的中心 center

$\|x - x_i\|$: x 和中心 x_i 之间的距离 (Euclidean norm)

ϕ : 径向基函数 (eg 高斯函数)

RBF 网络的最终输出 $F(x)$, 就是 N 个径向基函数 ϕ , 即曲面上山丘或涟漪的加权总和

Radial-Basis Functions

- Several widely used basis functions:

- **multiquadratics:** 多象限函数

$$\phi(x) = (x^2 + d^2)^{1/2} \quad \text{for some } d > 0$$

全局函数: 距离=0时, 它有最小值 d , 随着距离 x 增大, 函数值也随 x 增大。不像高斯那样衰减到0, 会受到远处数据点的影响。

- **inverse multiquadratics:** 反多象限函数

$$\phi(x) = 1 / (x^2 + d^2)^{1/2} \quad \text{for some } d > 0$$

局部函数: 当距离 $x=0$ 达到峰值1。随着 x 增大, 分母变大, 函数平滑地衰减并趋于0。

- **Gaussian:**

$$\phi(x) = \exp(-x^2 / 2\sigma^2) \quad \text{for some } \sigma > 0$$

当距离 $x=0$ 时, 达到峰值 $\exp(0)=1$ 。随着距离 x 增大, 它会非常快地衰减并趋近于0。G (sigma) 参数控制钟形曲线的胖瘦

Gaussian Function

高斯 RBF 的函数值，随着与中心点距离的增加而“单调递减”

A Gaussian RBF monotonically decreases with distance from the center. Gaussian-like RBFs are local (give a significant response only in a neighborhood near the center) and are more commonly used.

类高斯 RBF 是“局部”的（即：只在靠近中心的“邻域”内才会给出显著响应）

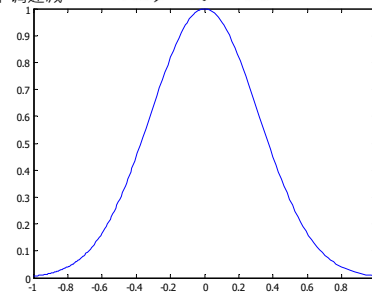
They are also more biologically plausible because their response is finite.

局部” (local) 意味着这个 RBF 神经元是一个“局部探测器”，它只关心那些与它自己的“中心点 c ”非常接近的输入 x 。如果输入 x 离它的中心 c 很远，这个神经元就基本“不激活”（输出值接近 0）。这使得 RBF 网络非常高效，因为它允许网络将一个复杂的问题分解为多个“局部”的小问题来解决。

r 越大：曲线越宽，会对更大范围内的输入 x 产生响应

r 越小：曲线越窄，只对离中心 c 极近的输入 x 才有响应。

钟形曲线 radius 定义了曲线展度



$c = 0$ and radius $r = 1$

$$h(x) = \exp\left(-\frac{(x-c)^2}{r^2}\right)$$

$h(x)$: RBF 激活值 (山丘在 x 点的高度)

x : 输入值 c : RBF 神经元的中心

6

正则化网络

Regularization Networks

- The regularization network models the interpolation function F as a linear superposition of multivariate Gaussian functions, which number is equal to the number of the given input examples N :

$$F(x) = \sum_{i=1}^N w_i \exp(-||x - x_i||^2 / 2\sigma_i^2)$$

where: w_i are the weights.

7

“插值函数 F ” (interpolation function F)：这就是 RBF 网络的最终输出 $F(x)$ ，也就是我们想拟合的那条“最佳曲线”或“最佳曲面”。

“线性叠加” (linear superposition)：这是一个物理和数学上的术语，你完全可以把它理解为“加权求和” (weighted sum)。

“多变量高斯函数” (multivariate Gaussian functions)：这就是我们一直讨论的“高斯山丘”。“多变量”仅仅意味着输入 x 可以是多维的（比如 [面积, 房龄]），而不只是一个单独的数字。

“RBF 的隐藏层的神经元节点数量等于给定输入样本的数量 N ” (which number is equal to ... N)：

它规定了这种 RBF 网络的隐藏层结构。

如果你有 N 个训练数据点（比如 1000 个房屋样本），那么你的 RBF 隐藏层就必须正好有 N 个神经元（1000 个高斯函数）。

第 i 个高斯函数（第 i 个神经元）的中心 x_i ，就直接被设定为第 i 个训练数据点的位置。

在这个模型里，你不需要用 K-Means 聚类或其他算法去“寻找”中心点 c 了。你直接把每一个训练数据点都当作一个 RBF 神经元的中心。

$$F(x) = \sum_{i=1}^N w_i \exp\left(-\frac{\|x - x_i\|^2}{2\sigma_i^2}\right)$$

$F(x)$ ：网络的预测输出

x ：输入的数据点

$\sum_{i=1}^N$ ：对 N 个神经元求加权总和

x_i ：第 i 个神经元的中心，第 i 个训练的数据点

w_i ：第 i 个神经元的权重

σ_i ：第 i 个曲线的宽度

$\exp(\dots)$ ：高斯函数

(径向基函数 RBF)

固定中心和宽度：学习过程

中心 x_i ：直接用 N 个训练数据点。（已固定）

宽度 σ_i ：通常会使用某种“启发式规则”来预先设定好（比如，所有 σ_i 都设为同一个值，这个值可能与数据点之间的平均距离有关）。（已固定）

求解权重 w_i ：

由于 RBF 的“形状”（中心和宽度）都已固定，整个网络 $F(x)$ 对 w_i 来说就成了一个线性模型。

学习任务就简化为：找到一组最佳的 w_i ，使得总误差最小。

无论你想“拟合”或“学习”的那个“真实规则” $f(x)$ 有多复杂（比如一个奇形怪状的曲面），只要它是“连续”的（即没有突然的断裂或跳变），RBF 网络都能模拟它。



$$F(x) = \sum_{i=1}^N w_i \exp(-\|x - x_i\|^2 / 2\sigma_i^2)$$

The important characteristic:

- it is a **universal approximator** in that it can approximate arbitrarily well any multivariate continuous function on a compact set, given a sufficiently large number of units.

只要 RBF 的隐藏神经元数量足够多，就可以让拟合误差尽可能地小，让拟合更精确。

Problems with the Regularization Networks

在实际情况下,寻找线性基函数权重时,需要对一个非常巨大的 $\Phi^T \Phi$ 矩阵求逆

高斯函数的数量与“训练样本的数量”之间“一一对应”的这种方式,通常在计算上是极其低效的

- The one-to-one correspondence between the number of Gaussian functions and the number of the training examples often become **computationally inefficient**, in the sense that it may require combining a very large number of basis functions (since often a large number of training examples are provided).
- In practical situations finding of the linear basis function weights needs to invert very large $\Phi^T \Phi$ matrices.
- In order to avoid such problems the **network topology should be reduced**. 为了避免计算问题,我们需要妥协,来减小网络的拓扑,即隐藏层的神经元数量 M 必须远远小于训练数据 N .
- This means that we will attempt to find a solution that **approximates the solution produced by the regularization network**. 但这样妥协的结果是我们只能得到一个近似解。

RBF Network Structure

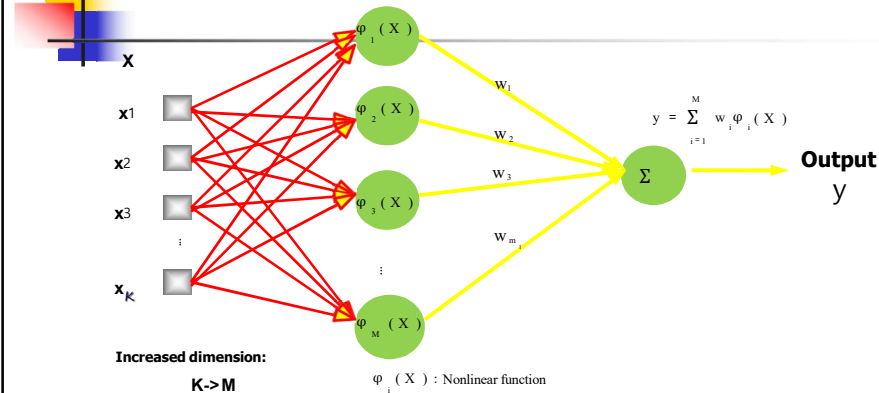
输入层: 不做任何计算, 将样本向量直接传递给下一层

- An RBF network architecture consists of 3 layers:
 - **input layer:** which passes the example vectors to the next layers
 - **hidden layer:** applies a **non-linear transformation function** to the inputs, and expands them in the usually very high-dimensional hidden space
 - **output layer:** applies a linear transformation to the given vector

这个“非线性变换函数”就是径向基函数 (RBF), 比如高斯函数。变换指的是: 它不再看 x 本身, 而是计算 x 与各个中心 C_i 的距离 $\phi_i(x) = \exp(-\|x - C_i\|^2 / 2\sigma_i^2)$

假设输入 x 是2维的, 隐藏层有500个神经元 (当2维数据传入之后, 每个神经元都计算 $\phi_i(x)$) 然后最后在输出层, 计算这500个 $\phi_i(x)$ 与 $[w_1, \dots, w_{500}]$ 的加权总和。

Radial Basis Function Network



- **Hidden layer:** applies a non-linear transformation from the input space to the hidden space.
- **Output layer:** applies a linear transformation from the hidden space to the output space.

11

当样本被投射到更高维空间时，它们在那里变得“线性可分”的可能性，远大于在低维空间中。

输入层 → 隐藏层 (非线性): 负责升维, 把线性不可分的低维数据, 映射成线性可分的高维数据
隐藏层 → 输出层 (线性): 负责在高维空间内, 执行“ $n \times n$ ”来判别类别

RBF Network: the Rationale

我们可以用一个非线性函数 (即 RBF), 将一个困难的非线性插值问题, 转变成一个更容易的、只涉及线性映射的问题。

即求线性加权和 $\sum w_i \phi_i$

- **Rationale behind RBF:** when the examples are cast in higher-dimensional space they are more likely to be linearly separable there than in a lower-dimensional space. RBF 网络被设计为: 执行从输入空间到隐藏空间的非线性映射, 然后再执行从隐藏空间到输出空间的线性映射。
- A nonlinear function can be taken to transform a difficult nonlinear interpolation problem into an easier one that involves linear mapping.
- That is why, the RBF networks are designed to perform nonlinear mappings from the input space to the hidden space, and next a linear mapping from the hidden space to the output space.

12

Function Mapping by RBF Networks

RBF 网络将插值函数 F 建模为“有限数量的多变量高斯函数”的“线性叠加”（即加权求和）

- The radial-basis function network models the interpolation function F as a linear superposition of a **finite number of multivariate Gaussians** as follows: 这里的 t_i 就是每个高斯函数的中心, 和之前的 x, c 意思相同.

$$F(x) = \sum_{i=1}^M w_i \exp\left(-\frac{\|x - t_i\|^2}{2\sigma_i^2}\right)$$

函数的数量 M 小于 训练集中的数据点数量 N

where: w_i are the weights, and the number of basis functions M **is less than** the number of points in the training set, i.e., **$M < N$** .

- The basis function centers t_i are determined in advance. 基函数的中心 t_i 是预先确定的.

13

阶段1: 确定中心 (非监督学习)

- 只关注输入的数据 x , 不看目标 y . (N 个数据点 x_i , M 个隐藏单元, $M \ll N$)
- 通常采用无监督算法 (例如 k-means) 从 N 个数据点中找出 M 个最有代表性的聚类中心.
- 这 M 个聚类中心 就被固定为 RBF 隐藏层的中心 t_i (宽度 σ_i 通常也是在这一步确定)

阶段2: 求解权重 (有监督学习)

- 一旦所有的中心 t_i 和宽度 σ_i 都固定了, RBF 网络就变成了一个线性模型
- 我们现在使用“有监督”方法 (最小二乘法 $\bar{w}_{opt} = \phi^T d$) 一次性解出使误差最小的最佳权重.

构建 $N \times M$ 的激活矩阵 Φ , $\Phi W = d$, 伪逆: $W = (\Phi^T \Phi)^{-1} \Phi^T d$

阶段1 中, 对 N 个输入样本 x_i 运行 k-means 算法, 帮助找到 M 个聚类中心, 用来当作 RBF 网络中的 t_i .

一旦 RBF 网络的中心 t_i 和宽度 σ_i 固定了 (在“阶段 1”中, 比如用 K-Means 聚类确定了), 我们不需要使用缓慢的“梯度下降”来迭代训练权重。

我们可以使用一个直接的数学公式来一次性解出“最佳权重”, 这个方法就是“伪逆法”

RBF Learning

- Weights can be computed by the **pseudo-inverse** method.

For an example (x_i, d_i) , consider the output of the network

$$o(x_i) = w_1 \phi_1(\|x_i - t_1\|) + \dots + w_M \phi_M(\|x_i - t_M\|)$$

$o(x_i)$ 是 RBF 网络对 x_i 的实际预测值

We would like $o(x_i) = d_i$ for each example, that is 我们希望针对每个输入样本 x_i 的输出值 $o(x_i) = d_i$

$$w_1 \phi_1(\|x_i - t_1\|) + \dots + w_M \phi_M(\|x_i - t_M\|) = d_i$$

This can be re-written in **matrix form** for one example 矩阵形式

$$[\phi_1(\|x_i - t_1\|) \dots \phi_M(\|x_i - t_M\|)] \cdot [w_1 \dots w_M]^T = d_i$$

所有 M 个隐藏神经元的激活值 ϕ 权重向量 w^T

14

如果把所有 N 个输入的数据点打包在一起, 形成一个可以被求解的最终方程。

RBF Learning

一个样本 x_i 对应: (一行激活值) $\cdot$ (权重向量) = (一个答案 d_i)

有 N 个样本, 就堆叠 N 行形成 输入数据点数量 $\times$ 隐藏节点数量的 matrix

$$\begin{bmatrix} \phi_1(\|x_1 - t_1\|) \dots \phi_M(\|x_1 - t_M\|) \\ \vdots \\ \phi_1(\|x_N - t_1\|) \dots \phi_M(\|x_N - t_M\|) \end{bmatrix} [w_1 \dots w_M]^T = [d_1 \dots d_N]^T$$

N M

for all the examples at the same time.

Let $\Phi = \begin{bmatrix} \phi_1(\|x_1 - t_1\|) & \dots & \phi_M(\|x_1 - t_M\|) \\ \vdots & & \vdots \\ \phi_1(\|x_N - t_1\|) & \dots & \phi_M(\|x_N - t_M\|) \end{bmatrix}$

看一行 (第 i 行): 就是把第 i 个输入数据 x_i 喂给所有 M 个隐藏神经元 (t_1 到 t_M) 后, 得到的那 M 个激活值 ϕ

then we can write $\Phi \begin{bmatrix} w_1 \\ \vdots \\ w_M \end{bmatrix} = \begin{bmatrix} d_1 \\ \vdots \\ d_N \end{bmatrix}$

看一列 (第 j 列): 就是所有 N 个输入数据 (x_1 到 x_N) 在第 j 个 RBF 神经元 (中心 t_j) 上的激活值, 15

RBF Learning

- Φ is the $N \times M$ design matrix of basis functions

代入高斯函数进行计算

$$\phi = \exp\left(-\frac{\|x - t_i\|^2}{2\sigma_i^2}\right)$$

$$\Phi = \begin{bmatrix} \phi_1(\|x_1 - t_1\|) & \dots & \phi_M(\|x_1 - t_M\|) \\ \vdots & \ddots & \vdots \\ \phi_1(\|x_N - t_1\|) & \dots & \phi_M(\|x_N - t_M\|) \end{bmatrix}$$

16

Radial-Basis Functions

既然已知 $\Phi W = d$
那么可以通过内蕴的方式
找到能得到最佳解 d 的
使均方误差最小的 W .

$$\Phi W = \Phi \begin{bmatrix} w_1 \\ \vdots \\ w_M \end{bmatrix} = \begin{bmatrix} d_1 \\ \vdots \\ d_N \end{bmatrix} = d$$

The unknown weights can be found by solving the following linear matrix equation using pseudo-inverse:

$$W = (\Phi^T \Phi)^{-1} \Phi^T d$$

where: W is the weight vector, d is the response vector,
and Φ is the **design matrix**

W : 要求的最优权重向量
 $(\dots)^{-1}$: 矩阵求逆
 Φ^T : 代表 Φ 矩阵的转置
 $\Phi^T \Phi$ 是 $M \times M$ 矩阵
 $(\Phi^T \Phi)^{-1} \Phi^T = \Phi^+$

17

RBF Learning

If Φ^+ is the pseudo-inverse of the matrix Φ we obtain the weights using the following formula

$$[w_1 \dots w_M]^T = \Phi^+ [d_1 \dots d_N]^T$$

$$\Phi W = d$$

$$W = \Phi^+ d$$

$$\Phi^+ = (\Phi^T \Phi)^{-1} \Phi^T$$

Pseudo inverse

Φ has a unique pseudo-inverse Φ^+ satisfying the conditions:

$$\Phi \Phi^+ \Phi = \Phi$$

$$\Phi^+ \Phi \Phi^+ = \Phi^+$$

$$\Phi^+ \Phi = (\Phi^+ \Phi)^T$$

$$\Phi \Phi^+ = (\Phi \Phi^+)^T$$

18

RBF Training Algorithm

- **Initialization:** Examples $\{(\mathbf{x}_e, d_e)\}_{e=1}^N$
- **Determine:**
 - the network structure with a number n of basis functions $\phi_i, i=1,2,\dots,M$
 - the basis function centers $\mathbf{t}_i, i=1,2,\dots,M$
 - e.g.: using **k-means clustering** algorithm
 - the basis function variances $\sigma_{i2}, i=1,2,\dots,M$

σ_i 可以设为 $\mathbf{t}_i$ 到它最近的邻居中心的距离 (也可以是其他的启发式规则)

19

RBF Training Algorithm

■ Compute:

- the outputs from each example with the Gaussian basis functions:

$$= \exp\left(-\frac{\|x - t_i\|^2}{2\sigma_i^2}\right)$$

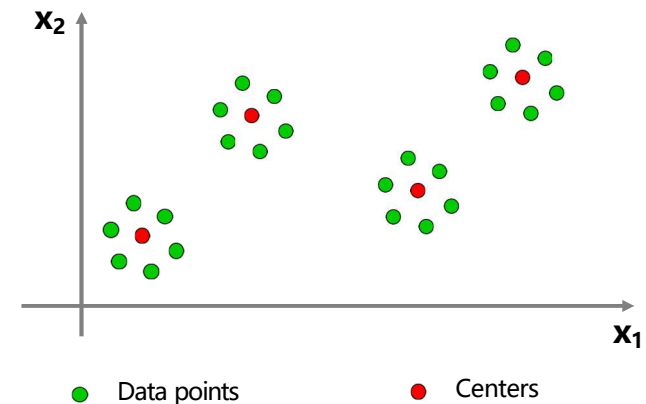
// at each i-th hidden unit where $i=1,2,\dots,M$; $e=1,2,\dots,N$

- the correlation matrix: $\Phi^T \Phi$ and pseudo inverse $(\Phi^T \Phi)^{-1}$
- the vector: $\Phi^T d$
- the weights: $W = (\Phi^T \Phi)^{-1} \Phi^T d$

Note: The radial-basis function centers t_i and variances σ_i^2 may be trained along with the output weights but this takes a long time. Thus they are often determined in advance.

20

RBF - Center Selection



21

Finding RBF Centers

■ Random Selection

- The simplest approach is to choose the centers randomly from the given training examples.
- If the training examples are distributed in representative way this approach gives good results.

22

使用 K-means 算法来选择中心 t_i 依赖于这样一个假设：相似的输入应该会产生相似的输出，因此这些相似的输入对应该被捆绑到训练集中的‘簇’里。

Finding RBF Centers

为一群近似结果的点，找一个中心代表，使它们形成一个聚集的簇。

■ K-means Clustering

K-Means 算法的本质就是去寻找“数据密度高”的区域的“质心”

- The use of the K-means clustering algorithm for the selection of the centers t_i relies on the assumption that similar inputs should produce similar outputs, and these similar input pairs should be bundled together into clusters within the training set. RBF 本身也只关注于局部，K-means 自动帮 RBF 把中心放在数据密集的地方。
- The K-means clustering places the RBF centers only in regions of the input space where there are significant amounts of training examples.

23

What is Cluster Analysis?

- Cluster: a collection of data objects
 - Similar to one another within the same cluster
 - Dissimilar to the objects in other clusters
- Cluster analysis
 - Finding similarities between data according to the characteristics found in the data and grouping similar data objects into clusters

24

What is Cluster Analysis?

- Clustering analysis is an important human activity
- Early in childhood, we learn how to distinguish between cats and dogs
- **Unsupervised learning**: no predefined classes
- Typical applications
 - As a **stand-alone tool** to get insight into data distribution
 - As a **preprocessing step** for other algorithms

25

Quality: What Is Good Clustering?

- A good clustering method will produce high quality clusters with
 - high intra-class similarity 内部的数据强相关
 - similar to one another within the same cluster
 - low inter-class similarity 外部的数据不相关
 - dissimilar to the objects in other clusters
- The quality of a clustering method is also measured by its ability to discover some or all of the hidden patterns 看是否能找到所有的潜在分布规律.

26

Similarity and Dissimilarity Between Objects

通过距离来衡量两个数据对象之间的相似性.

- Distances are normally used to measure the similarity or dissimilarity between two data objects i, j : 代表两个不同的数据点.
 - A popular one: Minkowski distance: p : 代表数据维度 (eg. 有3个特征, 那么 $p=3$).

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)}$$
 几何面积, 地段, 长度.
- where $i = (x_{i1}, x_{i2}, \dots, x_{ip})$ and $j = (x_{j1}, x_{j2}, \dots, x_{jp})$ are two p -dimensional data objects, q is a positive integer
- If $q = 1$, d is Manhattan distance

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$
 q : 横版参数 (可调整).
 - If $q = 2$, d is Euclidean distance:

$$d(i, j) = \sqrt{(|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 + \dots + |x_{ip} - x_{jp}|^2)}$$

27

K-means clustering algorithm

该算法将数据点划分(“分割”)成 K 个“不相交”的子集

- The algorithm partitions data points into K disjoint subsets.
- The clustering criteria: K是提前定义好的,在RBF中K=M
 - the cluster centers are set in the high density regions of data
 - a data point is assigned to the cluster with which it has the minimum distance to the center
- Mathematically, this is equivalent to minimizing the sum-of-square clustering function

$$J = \sum_{j=1}^K \sum_{n \in S_j} \| \mathbf{x}_n - \mathbf{c}_j \|^2$$

where

S_j : the j th cluster containing N_j data points

$\mathbf{c}_j = \frac{1}{N_j} \sum_{n \in S_j} \mathbf{x}_n$: the mean of the data points in cluster S_j

28



K-means clustering algorithm

Step 1: Initially randomly assign data points to one of K clusters. Each data points will then have a cluster label.

Step 2: Calculate the mean of each cluster C.

Step 3: Check whether each data point has the right cluster label.

For each data point, calculate its distances to all K centers. If the minimum distance is not the value for this data point to its cluster center, the cluster identity of this data point will then be updated to the one that gives the minimum distance.

Step 4: After each epoch checking (one turn for all data points), if no updating occurs, i.e, J reaches the minimum value, then stop; otherwise, go back to Step 2.

29

1. “分配”步骤 (Assignment step):

“a data point is assigned to the cluster with which it has the minimum distance to the center”

(一个数据点会被分配给那个“离它距离最近”的中心。)

遍历所有 N 个数据点。对于每一个数据点，你都要计算它到所有 K 个当前“中心点 c_j ”的距离（通常是欧几里得距离），然后把它“分配”给最近的那个中心。

2. “更新”步骤 (Update step):

“the cluster centers are set in the high density regions of data”

(簇的中心被设置在数据的“高密度区域”。)

在 (1) 中，所有的数据点都被“分配”给了某个中心。现在，对于每一个中心 c_j ，我们重新计算它的位置，把它移动到所有“分配”给它的那些数据点的“正中央”（即平均值或“质心”）。

这个算法就是不断地重复“（1）分配（2）更新 $\rightarrow$ （1）分配（2）更新...”，直到中心点的位置不再变化为止。

成本函数 J (总误差)

$$J = \sum_{j=1}^k \sum_{x_n \in S_j} \|x_n - c_j\|^2$$

$\sum_{j=1}^k$: 对所有 k 个簇 (从 1 到 k) 进行求和

$\sum_{x_n \in S_j}$: 对属于第 j 个簇 (S_j) 的每一个数据点 x_n 进行求和

$\|x_n - c_j\|^2$: 数据点 x_n 到它所属的中心 c_j 之间距离平方

成本 J 就是所有数据点到它各自的中心的平方距离总和

k -means 算法的目标就是找到一组 c_j (中心点)，使得这个总和 J 最小。

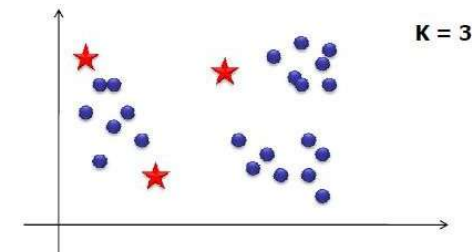
中心点 c_j : $c_j = \frac{1}{N_j} \sum_{x_n \in S_j} x_n$

第 j 个中心 c_j 的新位置，就是所有属于 S_j 簇的数据点 x_n 的平均值

(把它们加起来，再除以 S_j 簇的总点数 N_j)

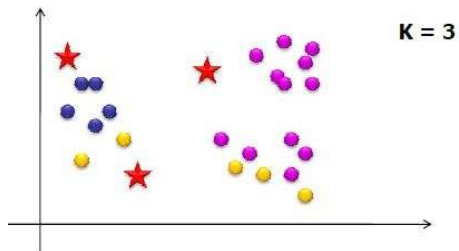
K-means algorithm description

- Choose the number of clusters - K
- Randomly choose initial positions of K centroids
- Assign each of the points to the “nearest centroid” (depends on distance measure)



K-means algorithm description

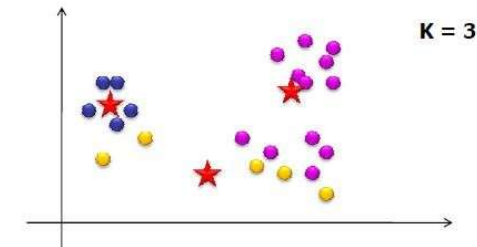
- Choose the number of clusters - K
- Randomly choose initial positions of K centroids
- Assign each of the points to the "nearest centroid" (depends on distance measure)



31

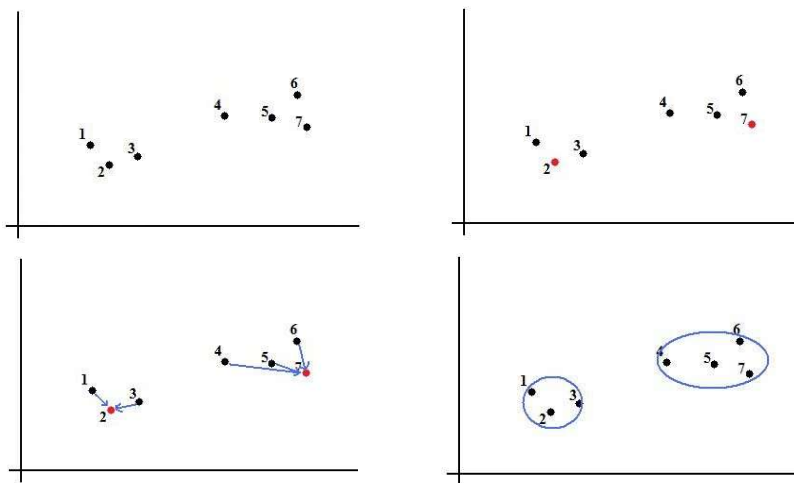
K-means algorithm description

- Choose the number of clusters - K
- Randomly choose initial positions of K centroids
- ▪ Assign each of the points to the "nearest centroid" (depends on distance measure)
- Re-compute centroid positions
- If solution converges Stop!



32

K-means Clustering



33

Issues

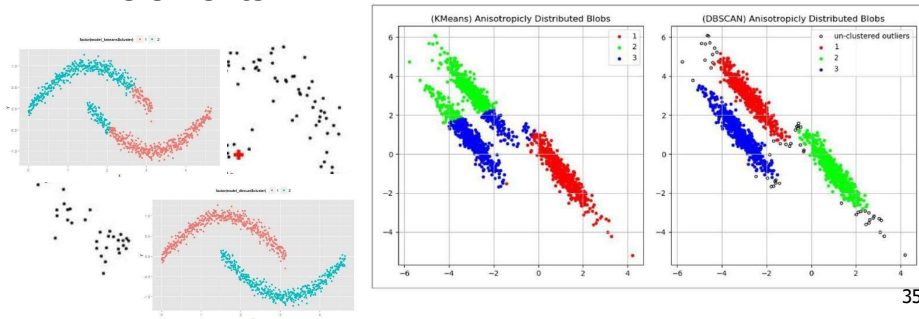
- The numbers of clusters must be specified in advance
- This method is not suitable for clusters with non-convex shapes 该方法不适用于非凸形的集群
- This method is sensitive to noise and outlier elements 对噪声敏感

k-means 方法
不适用的两种
情况

34

Issues

- This method is not suitable for clusters with non-convex shapes
- This method is sensitive to noise and outlier elements



35

Two ways to improve k-means

多次尝试不同的随机初始点 来进行更新

■ Repeated k-means

- Try several random initializations and take the best.

■ Better initialization 理想情况下: 利用更优的启发式方法来分配代码

- Use some better heuristic to allocate the initial 向量的初始分布 distribution of code vectors.
 - *Designing good initialization is not any easier than designing good clustering algorithm at the first place!*
- 但难度不亚于设计优秀的聚类算法.

36

RBF Networks vs. MLP

Similarities

- The RBF Networks and the MLP are **layered feedforward networks** that produce **nonlinear function mappings**;
- They are both proven to be **universal approximators**;

Differences

- An **RBF** network has **only one hidden layer**, while **MLP** networks have **one or more hidden layers** depending on the application task;
- The nodes in the hidden and output layers of MLP use the **same activation function**, while RBF uses **different activation functions** at each node (Gaussians parameterized by different centers and variances);
- The **hidden and output layers** of MLP are **both nonlinear**, while **only the hidden layer** of RBF is **nonlinear** (the output layer is linear);
- The activation functions in the RBF nodes compute the **Euclidean distance** between the input examples and the centers, while the activation functions of MLP compute **inner products** from the input examples and the incoming weights;
- MLP constructs **global approximations** while RBF construct **local approximations**.

37

- 都是前馈网络：两者都是“分层”的，并且信息“前向”传递（没有环路）。
- 都能非线性拟合：它们的主要工作都是“函数拟合”，即学习输入到输出的非线性映射关系（比如拟合复杂的曲线或曲面）。
- 都是“万能逼近器”：理论上都被证明，只要给予足够多的神经元，它们都能以任意精度逼近任何连续函数。

1. 网络深度（结构）：

RBF：通常只有一个隐藏层。

MLP：可以有一个或多个隐藏层（即“深度学习”中的“深度”）。

2. 输出层（计算）：

MLP：隐藏层和输出层都是非线性的（输出层通常也有一个激活函数，如 Sigmoid 或 Softmax）。

RBF：只有隐藏层是非线性的（高斯函数）；输出层是线性的（它只是一个简单的“加权求和”）。

$$\sum w_i \phi_i$$

3. 神经元激活方式（核心区别）：

MLP：激活是基于“内积” (inner products) 。 $\text{sigmoid}(\sum w_i x_i + b)$ ，它计算的是输入和权重的“加权总和”。

RBF：激活是基于“欧几里得距离” (Euclidean distance)，即 $\exp(-\|x - c\|^2 / 2\sigma_i^2)$ ，它计算的是输入 x 和中心 c 有多“近”。

4. 激活函数的“个性”：

MLP：所有隐藏神经元使用完全相同的激活函数（比如大家都是 ReLU）。

RBF：每个隐藏神经元使用不同的激活函数（因为每个高斯函数 $\exp(-\|x - c\|^2 / 2\sigma_i^2)$ 都有自己独一无二的“中心” c_i 和“宽度” σ_i ）。

5. 拟合方式（全局 vs 局部）：

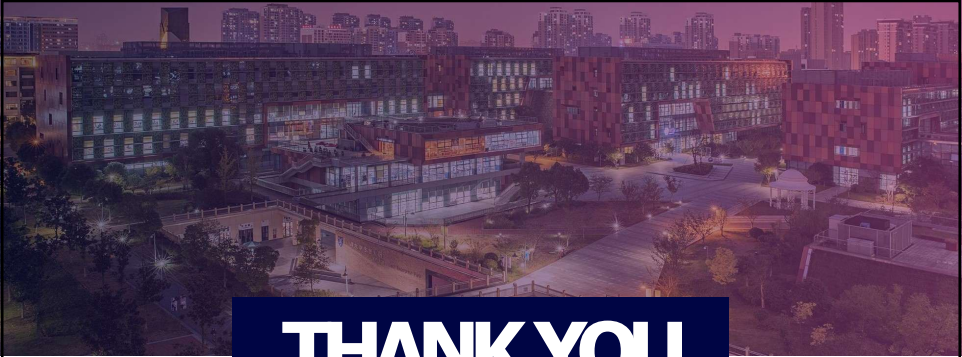
MLP：构建的是“全局”近似。一个输入会激活所有神经元，改变一个权重会（在某种程度上）影响整个函数的形状。

RBF：构建的是“局部”近似。一个输入只会激活它“附近”的少数 RBF 神经元（“高斯山丘”）。它像“打补丁”一样，用很多“局部”函数拼凑出“全局”函数。

RBF Training Algorithm

- **Initialization:** Examples $\{(\mathbf{x}_e, d_e)\}_{e=1}^N$
- **Determine:**
 - the network structure with a number n of basis functions $\phi_i, i=1,2,\dots,M$
 - the basis function centers $\mathbf{t}_i, i=1,2,\dots,M$
 - e.g.: using *k-means clustering* algorithm
 - the basis function variances $\sigma_i^2, i=1,2,\dots,M$
- **Compute:**
 - the outputs from each example with the Gaussian basis functions:

$$\phi_{ei} = \frac{\exp(-\|\mathbf{x}_e - \mathbf{t}_i\|^2)}{2\sigma_i^2}$$
 - the weights: $\mathbf{W} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{d}$



THANK YOU



VISIT US
WWW.XJTLU.EDU.CN



FOLLOW US
@XJTLU



Xi'an Jiaotong-Liverpool University
西交利物浦大学

39